# Mega Revisão de SO – ProfºDr.Leon

## Conceitos básicos de S.O

- O que é um sistema operacional?

Programa gerenciados de recursos computacionais

- O que é um recurso computacional?

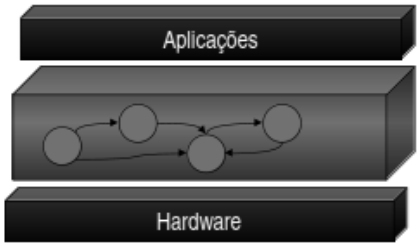
Um objeto necessário para efetuar

Sistema operacional é responsável pelo gerenciamento de CEP, memória e Dispositivos de E/S, pela criação e Gerenciamento de Sistemas de Arquivos, pela Segurança, e pela redes de comunicação.

### Arquitetura de Computadores

- Registrador de Proposito Geral: Armazenam valores temporários de qualquer tipo
- Registrador de Instrução: Armazena o endereço da instrução Atual
- Contador de Programa: indica qual será a próxima instrução a ser executada
- Stack Pointer: armazena o endereço de memória para o topo da pilha

- Sistemas monolíticos:
  - ✓ Todos os componentes do núcleo se inter-relacionam conforme suas necessidades e sem restrições de acesso.
  - ✓ Exemplos: versões do Linux anteriores à 2.0; DOS.
  - ✓ Vantagem: velocidade
  - ✓ Desvantagem: baixa robustez.




- Sistemas em camadas
  - ✓ A camada inferior realiza a interface com o hardware; as camadas intermediárias proveem níveis de abstração e gerência; a camada superior define a interface do núcleo para as aplicações.
  - ✓ Vantagem: maior abstração do sistema e aumento da robustez.
  - ✓ Desvantagem: redução da velocidade do sistema por causa do despacho do pedido entre as camadas.
  - ✓ Exemplos: Windows NT, OS/2.

- Micronúcleo
  - ✓ Apenas o código de baixo nível, para lidar com o hardware, é mantido no núcleo. Os demais módulos são executados como aplicações de usuários.
  - ✓ Vantagem: robustez e flexibilidade.
  - ✓ Desvantagem: baixo desempenho, uma vez que o sistema se baseia em troca de mensagens.
  - ✓ Exemplos: MacOSX.

27

## Gerenciamento de Processos

Ao gerenciamento do recurso CPU, em geral, trabalhamos com programas em execução chamados de processos.

O sistema operacional é responsável pelo escalonamento, agendamento do próximo processo, e pela troca de contexto. O responsável pelo escalonamento é o escalonador.

- A operação de transferência de controle da CPU de um processo para outro é o que chamamos **troca de contexto**

## Serviços Relacionados a Processos

- Criação de processo, destruição de processo, alteração de prioridade do processo, mecanismos de comunicação inter-processos, mecanismos de sincronização de processos, barramos outros processos para esperar outro terminar.

## Gerenciamento de Memória

- Atendimento a solicitação de alocação e liberação de memória.
  - Alocação dinâmica é dinâmica porque é alocada durante a execução
- O SO deve assegurar que os processos não interfiram uns com os outros e que o espaço de memória não seja desperdiçado.

## Serviços Relacionados a Memória

- Solicitar memória adicional diretamente
- Solicitar memória indiretamente (por exemplo, ao criar um novo processo)
- Liberar memória de volta para o SO
- Solicitar áreas de memória para serem compartilhadas entre processos

## Segurança

- Parte das responsabilidades de gestão de outros programas
- Autenticar a identidade de um solicitante
- Autorizar o acesso conforme a política de segurança

## Redes

Os serviços de rede incluem:

- Estabelecer uma conexão com um serviço remoto;

- Atender conexões de um cliente remoto;
- Enviar mensagens para um sistema remoto;
- Receber mensagens de um sistema remoto;
- Fechar uma conexão com um sistema remoto.

### Kernel ou Núcleo

Ele é parte intregante do sistema operacional, e é responsável pelas funções gerenciamento do sistema operacional

O núcleo do sistema operacional contém os módulos necessários para o gerenciamento do sistema computacional:

- Alocação de processos
- Escalonamento de processos
- Particionamento e alocação de memória
- Arbitragem de recursos

Núcleo: processador físico

Threads: Processadores Lógicos

## Hyperthreading

Nos anos 2002 a intel percebeu que trabalhando com múltiplos processos, eu tenho a chamada de troca de contexto e podemos executar simultaneamente para o usuário, na prática, o processador esta dedicando um de cada vez. Assim surgiu o tempo compartilhado, o processador se dedica a cada serviço a cada um, milissegundos geralmente, dessa forma surge a lentidão de abrir vários programas, porque há a divisão do tempo de processamento em n programas.

- Registrador de propósito específico: controla o contexto do processo atual.

Com a troca da informação do registrador de propósito específico, com isso há a **troca de contexto**.

A Intel percebeu que essa troca de contexto poderia ser sua velocidade aumentada, então ela duplicou o barramento interno e os registradores. Fazendo isso a intel chamou de **Hyperthread** e AMD chamou de **processador virtual**

- Registradores só armazenam dados e memória

## Multicore

Agora as fabricantes duplicaram o processador, a unidade de controle, a ULA, barramento e registradores, criando o conceito de multicore, e podemos ainda adicionar o processamento virtual a cada um deles e duplicando o poder de processamento.

# GPU - Processador Vetoriais

GPU são componentes muito mais simples, e executam múltiplas execuções simultaneamente, trabalhando muito bem com vetores e matrizes.

# Aula 2 - Sincronização

# Comunicação inter processos

Muitas vezes é importante que as tarefas/processos cooperem para a solução de um problema.

As tarefas podem compartilhar a mesma área de memória em um mesmo processo ( threads) ou em múltiplos processos

As formas de comunicação podem ser:

- Através de trocas de mensagens ou compartilhado a mesma área de memória

Obs: trocas de mensagens é mais ineficiente do que compartilhamento da área de memória, pois precisa criar uma rede com ela mesma. Para criar uma área de memória compartilhada é necessário pedir ao sistema, além do mais a área compartilhada deve ter seu id ou chave nos processos, e para ter acesso a essa área também precisa pedir para o SO, e necessita de uma referência/ponteiro para acessar essas informações.

Podem haver problemas de sincronização tendo um processo sobre escrevendo os dados dos outros

# Sincronização

Sincronizar é fazer com que dois ou mais processos não tenham acesso à uma região crítica ao mesmo tempo.

**Região crítica:** é a região onde os processos podem acessar devido relacionamentos

**Condição de corrida:** situação onde dois ou mais processos concorrem pelo mesmo recurso

**Exclusão mútua**: garantia que apenas um processo usará a região crítica em cada instante.

Quatro condições para prover a **exclusão mútua:**

- Jamaiss dois processos entrarão simultaneamente na mesma região crítica
- Nenhuma afirmação a respeito da velocidade dos processos
- Nenhum processo Dora da região crítica pode bloqueado

## Técnicas de sincronização

Ao entrar na região crítica o processo suspende as interrupções, m

Lrnenas a

**Semáforos**

Criados 1965, por E.Dinkstra, os semáforos são a técnica mais usada atualmente.

Um semáforo é uma variável (s), associada a uma região crítica, sobre a qual podem incidir duas operações:

Operação UP: incrementa o valor de s, e desbloqueia as demais tarefas se o valor for zero

Operação Down: verifica o calor de a é maior que zero. Se for, o valor é decrementado. Senão, a tarefa é bloqueado e o valor de a permanece 0

**Monitores:**

Garantem que apenas um processo está usando um procedimento ou função do programa em um dado instante.

Dessa forma, embute-se o acesso à região crítica em um procedimento ou função e assim evita-se que dois ou mais processos acessem a região em um instante

**Troca de mensagens:**

Mais úteis em sistemas distribuídos

Um processo fica bloqueado até receber uma mensagem de que pode usar a região critica

Ha um processo de controle, a qual gerencia a região crítica, os processos necessitam pedir permissão para acessa-la

**Barreiras:**

Bloqueia os processos até que todos tenham terminado seus processamentos

**Solução com espera computada**

Tentando continuamente conseistinod como

Existe três formas

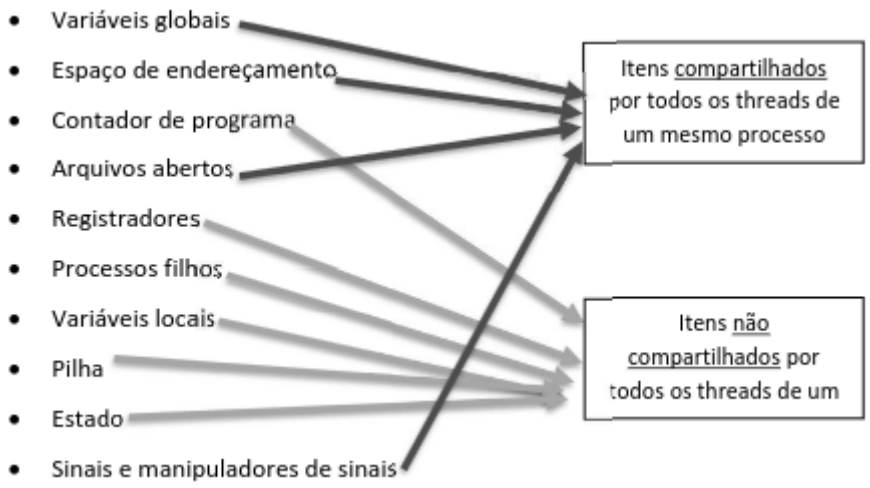
## Problemas de sincronizacao

Produtor/ consumidor: este problema refere-se, a um buffer sendo a região crítica, o produtor deve parar de produzir quando o buffer encher, e o consumidor esperar pelo produtor e não deixar vazio, e voltar a consumir quando tiver algum dado

# Aula 3

### Memoria

A memoria é subdividadas em paginas, entao para calcularmos o tamanho da memoria, pegamos o tamanho da pagina e multiplicamos pela quuantidade de paginas



# Algoritmos de Escalonamento

A função principal de um **escalonador** é gerenciar a execução de múltiplos processos em um sistema operacional, determinando qual processo terá acesso ao processador em um determinado momento. Ele atua como um árbitro, distribuindo o tempo de CPU entre os diversos processos que competem por este recurso limitado.

**Algoritmos Não Preemptivos:**

- **FCFS (First-Come, First-Served):** O processo que chega primeiro é o primeiro a ser executado. Simples e fácil de implementar, mas pode levar a longos tempos de espera para processos curtos se um processo longo chegar primeiro.
- **SJF (Shortest Job First):** O processo com menor tempo de execução estimado é executado primeiro. Ideal para minimizar o tempo de espera médio, mas exige estimativas precisas do tempo de execução.
- **Prioridade:** Cada processo possui uma prioridade associada. O processo com a maior prioridade é executado primeiro. A prioridade pode ser estática (definida no início) ou dinâmica (atualizada durante a execução).

**Algoritmos Preemptivos:**

- **SJF Preemptivo (Shortest Remaining Time First):** Similar ao SJF, mas permite que um processo com tempo de execução restante menor interrompa um processo em execução.
- **Round Robin:** Cada processo recebe um quantum de tempo para executar. Se o processo não terminar no quantum, ele é interrompido e outro processo é executado.
- **Prioridade com Preempção:** Similar ao algoritmo de prioridade não preemptivo, mas permite que um processo com prioridade mais alta interrompa um processo com prioridade mais baixa.
- **Multilevel Queue:** Divide os processos em múltiplas filas com diferentes prioridades. Cada fila pode ter seu próprio algoritmo de escalonamento.
- **Multilevel Feedback Queue:** Combina múltiplas filas com diferentes quantum. Os processos migram entre as filas com base em seu comportamento.

# Aula 4 e 5 - Processos Threads

Nesta aula foi abordado os principios do terminal linux, e alguns comandos como o **top, ps -ef, grep entre outros...**

## Processos zumbis

Processos zumbis são processos que já terminaram e por algum motivo aindam estão na tabela de processos. **Como são criados:** Eles são criados devido, a não percepção do pai que o processo filho já acabou, assim não avisando o sistema de sua conclusão.

Como podemos extermina-los? Sabendo o PID do pai podemos enviar um kill com o PID do pai e matarmos todos os filhos.

```
kill -s SIGCHLD pid
```

**Espaço de endereço do processo:** Conjunto de endereços que o sistema reserva para cada processo

O indetificador do segmento compartilhado deve ser maior que 1024 e menor que 32 mil e alguma coisa, menor que 1024 é reservado para o sistema

printf: direciona uma mensagem para o saída padrão

perror: direciona uma mensagem para saída padrão de erros

readelf -a: ele lê o arquivo em sua forma pura

Para determinar a “extensão” do arquivo em linux, ele utiliza-se de um magic number

*.o: extensão para object file

void *funcaoThread(void *arg);

neste caso void é qualquer tipo, para enviar vários argumentos envie-os por structs

pthreads exit informa que a thread que esta acabou e retorna o endereço da variavel de resultado, para o pthreads join ser avisado e o recebe

palavra-chave register: armazena a variavel no registrador geral, ao invés da memória, o registrador é 2000x mais rapido do que o memória

para copilar tudo para um único programa utiliza-se gc -pthread arq0.c arq1.c -o arq.out

# Aula 6 - DeadLocks

Um processo pode utilizar-se de um recurso, ou memória compartilhada, onde esses processor requerem recursos ou detém recursos

**O que é um grafo:** Um grafo é uma estrutura de dados, formadas por vertices e arestas, sendo estas arestas bidirecionais ou unidirecionais

**Grafos em Sistemas Operacionais:**

Em sistemas operacionais, grafos são frequentemente utilizados para modelar e analisar diversos aspectos, incluindo:

- Alocação de recursos: Representam a relação entre processos e recursos do sistema.
- Detecção de deadlocks: Ajudam a identificar situações de impasse entre processos.
- Escalonamento de processos: Modelam dependências e prioridades entre tarefas.
- Gerenciamento de memória: Ilustram a organização e alocação de espaço na memória.
- Roteamento de rede: Descrevem a topologia e conexões em redes de computadores.

Os grafos permitem uma visualização clara e eficiente dessas relações complexas, facilitando a tomada de decisões e a otimização do sistema operacional.

## Deadlock

Situação de deadlock, quando um processo que quer requerem um recurso, enquanto outro também quer dete-lo/ ja tem. Pode ser gerado quando um estado quer fazer uma requisição /alocação de um recurso ja em uso.

Quando tenho processos e recursos totalmente bloquedos, e não há nenhuma chance de conseguirem continuar sozinhos. Dessa forma o sistema se bloqueia

Estado seguro: é aquele que o sistema garante que qualquer sequencia de transições leva a outro estado seguro. Enquanto o inseguro é o contrario

### Caracterização de Deadlocks

Um conjunto de N tarefas esta em deadlock (impasse) se cada uma das tarefas aguarda um evento que somente outra tarefa do conjunto poderá produzir

Quatro condições são necessrárias para existir um deadlock:

1. Exclusão mútua: o acesso aos recursos deve ser feita de forma mutuamente exclusiva, apenas um processo por vez pode usar aquilo.
2. Posse e espera: Um processo está em posse de um recurso enquanto espera novos recursos, uma tarefa pode solicitar outros recursos sem ter que liberar os recursos já obtidos
3. Não-Preempção: uma tarefa somente libera os recursos que detém se desejar
4. Espera circular: existe um clico de tarefas, e uma está aguardando o que a outra detém

## Tratamento de DeadLocks

Para prevenir deadlocks, utilizaremos de alguma forma para combater uma das 4 condições:

- **Algoritmo do avestruz:**
- Processo gerenciador de recursos (spooler).
- Utilização de apenas um recurso por vez ou permitir que a tarefa só continue se já possuir acesso a todos os recursos que precisa.
- Preempção: retirada do recurso pelo SO a força.
- Quebrar a espera circular. Para quebra-la precisamos bloquear um desses processos,a fim de outro processo cumprir todas as suas necessidades de recursos, assim desencadeando o ciclo.

2) Considere o código a seguir escrito er

```
1:  mov x, 10h
2:  mov y, 20h
3:  cmp x, y
4:  jbe 7
5:  add x, 1h
6:  jmp 8
7:  sub y, 1h
8:  mov z, 0h
9:  add z, x
10: add z, y
```

4ª Questão

No UNIX, usando-se o comando `fork( )`, um novo processo é criado sem que um novo programa seja posto em execução. Assinale **V** (verdadeiro) ou **F** (falso) para as afirmações a seguir:

| | |
|---|---|
| (V) | Caso não ocorra falha, a chamada no sistema `fork( )`, retorna no mesmo instante o valor do PID do processo filho para o processo pai. |
| (F) | Se um processo B for criado por um processo A via `fork( )`, o processo B executará o programa que estiver sendo executado por A repetindo exatamente os mesmos fluxos. Embora ambos os processos estejam executando o mesmo programa, não significa que eles sejam idênticos. Os fluxos dos processos ainda podem ser distintos. |
| (F) | A mudança de variável global em um processo filho afeta o processo pai, pois os dados dos dois processos são iguais. Como todo processo tem seu próprio espaço de endereçamento, qualquer modificação será independente, isto é, a alteração em um processo não se refletirá no outro. |
| (V) | Os valores iniciais das variáveis do processo filho são iguais às do processo pai no momento da execução `fork( )`. |

Começando da primeira instrução, para cada ciclo de instrução (→) escreva o conteúdo dos dois registradores especiais - Contador de Programas (CP) e Registrador de Instruções (RI).

R. **Contador de Programas**

Início: 2 → 3 → 4 → 7 → 8 → 9 → 10 → 11 → Fim.

**Registrador de Instruções**

Início: mov → mov → cmp → jbe → sub → mov → add → add → Fim.

**Conteúdo final das variáveis:** x = 10; y = 19; z = 29.

- (A|O) Shell é um dos módulos do sistema operacional responsável pela interação com o usuário pela linha de comando.
- (A|O) Kernel é responsável pelo gerenciamento de processos.
- O Registrador de Instruções é um registrador de propósito específico responsável pelo armazenamento da instrução atual.
- (A|O) Heap é local da memória que armazena as variáveis alocadas por um processo.
- As chamadas de função são armazenadas (na|no) Pilha.

Contador de Programas | Gerenciador de Janelas | Tabela de Processos

Área de código | Escalonador

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_0$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_1$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_2$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_3$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_4$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_5$ | | | | | | | | | | | | | | | | | | | | | | | | | |

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_0$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_1$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_2$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_3$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_4$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_5$ | | | | | | | | | | | | | | | | | | | | | | | | | |

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_0$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_1$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_2$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_3$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_4$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_5$ | | | | | | | | | | | | | | | | | | | | | | | | | |

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_0$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_1$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_2$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_3$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_4$ | | | | | | | | | | | | | | | | | | | | | | | | | |
| $P_5$ | | | | | | | | | | | | | | | | | | | | | | | | | |

```
Semáforos mutex_garcom = 0 , mutex_mesa = 0 , mutex_participantes = 1 ;
Inteiro quitutes = Q;
```

**Thread Participante (int id)**

```
Início
  while (true)
  {
      down(mutex_participantes);
    if (quitutes == 0)
    {
      up(mutex_garçom);
      down(mutex_mesa);
    }
    else
      quitutes = quitutes - 1;
      up(mutex_participantes);
  } // fim while
Fim.
```

**Thread Garçom ()**

```
Início
  while (true)
  {
      down(mutex_garçom);
    quitutes = Q;
      up(mutex_mesa);
  }
Fim.
```